EVA: Step-by-Step Setup

From the project folder to your first conversation

Updated 29 October 2026: menu, voices, startup and VR controls.

Start with the complete EVA project folder on Windows with an NVIDIA card. Test desktop mode before adding a headset. Python connects the speech tools; Unity displays the character, records your voice and plays the answer.

Step 1: check the folder

Keep Unity closed during first setup. Example location:

```
C:\Projects\EVA
```

Replace this example path with your project location. Keep these complete folders: `Assets`, `Packages`, `ProjectSettings` and `Backend`. Also keep `README.md`.

Copied from another PC? Recreate `Backend/.venv` in Step 4 with local Python 3.11 x64. A copied environment can fail with *No Python at* / exit 103. With Unity closed, discard a copied `Library` cache; Unity recreates it. Keep model, voice and project files.

If you clone the GitHub repository instead, use Git LFS to obtain the avatar model and texture files:

```
git lfs install
git clone https://github.com/oxi-diert/EVA.git
cd EVA
git lfs pull
```

These commands are unnecessary if the complete folder was already copied.

Step 2: check the target PC

Open PowerShell in the project's `Backend` folder:

```
cd 'C:\Projects\EVA\Backend'
Set-ExecutionPolicy -Scope Process Bypass
.\hardware_report.ps1
notepad .\hardware-report.json
```

Check the GPU, graphics memory, driver, RAM, Windows version and free storage. An old copied report may describe another PC.

This version requires NVIDIA/CUDA. It has no CPU, AMD or Intel fallback. The desktop example uses an RTX 5070 Ti with 16 GB graphics memory and 32 GB RAM; the laptop example uses an RTX 5070 Laptop GPU with about 8 GB graphics memory. These are examples, not measured minimum requirements. Allow at least 12 GB free for backend setup, plus Unity, assets and the import cache.

Step 3: install the basic software

Install missing items. Use Windows x64 versions.

1. **NVIDIA driver.** Install a suitable current driver from the official NVIDIA driver page. Restart Windows if requested, then rerun the hardware report.
2. **Python 3.11 x64.** Use the official Windows downloads. Choose an installer or install manager that provides this version. An existing working Python 3.11 x64 installation is sufficient.
3. **Git for Windows with Git LFS.** Get it from git-scm.com. Make Git available on the command line. Unity needs it for NativeWebSocket; Git LFS retrieves avatar assets.
4. **Unity Hub and Unity 6000.3.12f1.** Install Unity Hub, then this editor version through Hub or Unity's archive. Activate a Unity license valid for your use. Newer editor versions have not been tested with this project.
5. **Microsoft Visual C++ x64 Runtime.** Use Microsoft's supported Redistributable download. It supplies native Windows libraries needed by the GPU software.
6. **Ollama.** Step 4 can install it with Windows Package Manager (winget). Alternatively, use the official Windows installer and keep the normal per-user location.

Check the installations

Close and reopen PowerShell so it sees newly installed programs:

```
py -3.11 --version
py -3.11 -c "import struct; print(struct.calcsize('P')*8)"
git --version
git lfs version
nvidia-smi
```

Python must report `3.11` and `64`. Git and Git LFS must print versions. `nvidia-smi` must show the NVIDIA card and driver. If the Python launcher is missing, follow the Python Windows guide. Fix missing programs first. A newer Python major/minor version is not a replacement for the version required by this project.

Step 4: install the backend and its models

In the new PowerShell window:

```
cd 'C:\Projects\EVA\Backend'
Set-ExecutionPolicy -Scope Process Bypass
.\setup_backend.ps1 -RecreateEnvironment -InstallOllama
```

Use this for first setup on a new PC. It replaces only the old Python environment, not `Backend/models` or `Backend/voices`. Internet is needed for missing packages and downloads.

Setup checks NVIDIA support, creates a Python 3.11 x64 environment, installs the pinned packages, supplies CUDA runtime libraries, checks/downloads Whisper Small and all four Piper voices, installs/checks Ollama and `llama3.2:3b`, and runs preflight. Let it finish:

```
[EVA Setup] SUCCESS: The EVA backend is ready.
```

If winget is missing, install Ollama manually and rerun setup. Check the cause of a setup failure before continuing. A separate CUDA developer toolkit, PyTorch installation and FFmpeg executable are normally unnecessary.

Step 5: check the backend installation

Stay in `Backend`:

```
.\.venv\Scripts\python.exe verify_backend.py
```

The last line must be:

```
[EVA Check] SUCCESS: The local GPU backend is ready.
```

The output should include `CUDAExecutionProvider`, a CTranslate2 CUDA device, FP16 support and the expected model files. Full model warm-up happens when Unity starts the backend. Calling the environment's Python directly avoids using a different Python by mistake.

Optional automatic checks

These do not record a microphone or load AI models:

```
.\.venv\Scripts\python.exe -m unittest -v test_response_language
   test_voice_preview test_streaming_transcription
```

To generate test WAVs and check all four real CUDA voices, run this **outside Play**. It loads its own voice models:

```
.\.venv\Scripts\python.exe test_voice_profiles.py
```

The live tests in Step 9 use an already running backend instead.

Step 6: open the existing Unity project

In Unity Hub, add/open the folder containing `Assets`, `Packages` and `ProjectSettings`. Use Unity **6000.3.12f1**. Wait for import, then open `Assets/_Project/Scenes/Main.unity`. This is the configured build scene; the old Unity example scene was removed. Fix compiler errors first. Before Play, choose EVA > Controls > Desktop (WASD + Mouse).

The saved scene has the required references. If the menu's avatar host needs assigning, select the intended EVA root, run EVA > Interface > Configure Selection Menu in Scene, and save with Ctrl+S. Only one avatar host is needed; existing extras are hidden during Play. Object names may differ. The menu's Primary Avatar reference matters.

Step 7: allow and choose the microphone

In Windows Settings > System > Sound, select the input microphone and output speakers/headphones. Speak and check the input level. In Windows 11 Settings > Privacy & security > Microphone, enable microphone access and access for desktop apps. See Microsoft's microphone guide. An empty Unity Microphone Device field uses the Windows default. Restart Play after changing devices.

Step 8: choose your EVA and start talking

1. Press Play. Do not start another server manually. The launcher starts or reuses Ollama and Python, warms models and waits for `server.ready`. Recording and movement stay blocked while loading and while selecting.
2. Choose **01 CHARACTER** (six designs) and **02 BEHAVIOUR** (Neutral, Friendly or Expressive body animations). One character stays visible.
3. Set **03 LANGUAGE** to English. It fixes recognition, answer text and voice to English. EVA does not detect or change languages automatically.
4. Under **04 VOICE**, click Kristin or Mike to select a voice. The small play icon in either card previews without selecting it. Click its loading/stop icon again to cancel or stop.
5. Select **Start conversation**. Click empty Game-view space, hold **B**, ask *What can you do?*, and release B to send.
6. Text boxes start hidden. Press **T** to show the recognized question and answer. Words appear during recording and answer generation. Recognition has a short delay and may correct recent words. Listen to the full answer and check lip-sync.

Confirmed choices are saved locally. Previews stop when you change language/voice, start conversation or leave the menu.

Step 9: check controls, language and voice

Control	Action
W/A/S/D	Move at the current camera height.
Mouse / Shift	Look around / move faster.
Esc	Release the mouse for menu and text-box controls.
B, hold/release	Record/send; a new recording interrupts the old answer.
T	Show/hide text boxes; use their move/resize handles with the mouse released.
M	Open settings while idle; continue from the open menu.

When EVA is idle, press M. Select **Deutsch** and Ramona or Thorsten. Preview them, select **Continue conversation**, then ask *Was kannst du machen?* EVA should recognize and answer in German. Change language only through the menu. Voice is independent of design and stored separately for each language.

Try all styles and several designs. Walk around EVA to check head/body tracking. A preview only plays audio; it does not animate the mouth or add conversation text. Normal answers still use the 15-viseme lip-sync system.

With the backend ready, PowerShell in `Backend` can run:

```
.\.venv\Scripts\python.exe test_live_voice_preview.py
.\.venv\Scripts\python.exe test_live_voice_profiles.py
```

The second command needs the WAVs generated in Step 5. These clients test their own connections, not Unity's speakers or microphone.

Step 10: add Meta Quest 3 if needed

1. Stop Play. Install the Meta Quest Link PC app, set it as the active OpenXR runtime and start a USB Link or Air Link session.
2. Choose EVA > Controls > VR (Meta Quest) before Play. After loading, use the right stick and A for the menu.
3. After starting conversation, release A before holding it again to record. The left stick moves; the right stick snap-turns. Right B opens settings while idle or continues from the menu. Keyboard M also works.
4. Check headset microphone/output settings and repeat both language tests.

This is PC VR: Unity and the models run on Windows, not on the headset.

Controller buttons

Control	Action
Right stick in menu	Move between cards, preview icons and Start/Continue.
Right A in menu	Confirm the selected control. Release A before recording.
Right B	Open settings while EVA is idle; Start/Continue closes the open menu.
Right A, hold/release	Record the question / finish recording and send.
Left stick	Walk during conversation.
Right stick	Snap-turn by 30 degrees during conversation.
Left X	Show/hide the question and answer text.
Left Y	Place the menu, text and loading surfaces again from your current position.

Where the VR interface appears

The menu sits beside EVA. Conversation text sits below the face, and loading appears in front of you. The surfaces are usually about 2–3 m away. They stay in the scene when you turn your head. VR text starts visible; its saved visibility is separate from the desktop layout. Use X to hide it and Y after moving to reposition the surfaces. Opening the menu also places it again.

To adjust size and distance, select `Assets/_Project/Resources/EVAControlSettings.asset` in the Project window. In the Inspector, find VR panels (metres). Change distance, menu/text width, menu side offset or text vertical offset. Press left Y to apply changes during Play. Edit and save the asset outside Play to keep values for next time.

Check both eyes, standing and seated. Menu and text should be readable near EVA. Test X, Y and B, then choose menu controls with the right stick and A. Loading and selection must still block recording and walking. Release the confirmation press before holding A to speak.

Next time

Open Main, choose Desktop or VR, press Play, wait, select settings and start. Start Quest Link first for VR. Do not reinstall packages each day. Stop Play before editing scripts. Unity stops the process trees it started; independent services remain running. Restart Play after compilation.

If a step fails

What you see	What to check
No Python at / exit 103	The copied environment may name the old PC's Python. Stop Play, install local Python 3.11 x64, repeat Step 4 and verify. Keep the model and voice folders.
Git or nvidia-smi missing	Install Git or the NVIDIA driver. Reopen PowerShell; restart Windows if requested.
Ollama missing / winget missing	Install Ollama manually, then rerun setup. For a non-standard location, use Ollama Executable Override.
CUDA provider / DLL error	Use the project Python and rerun setup. Do not mix CPU `onnxruntime` and `onnxruntime-gpu`. Check the driver and Visual C++ x64 Runtime.
Download or Unity import fails	Check network access, Git LFS and editor version. Rerun setup for missing models; fix compiler errors before Play.
Loading / WebSocket error	Read Unity's first error and `Backend/backend-runtime.log`. An open port is not enough: Unity waits for the server's readiness message. Fix the cause and restart Play.
Immediate close / error 1011	This indicates a server-side failure. Check the runtime log. An old EVA process can survive an interrupted reload; stop only that confirmed EVA process tree, then restart. Keep unrelated Python apps.
Missing CanvasRenderer	Update the preview scripts, finish compilation and restart Play. The icon requires a renderer and the menu creates it explicitly. No manual addition to a runtime object is needed.
Recording does nothing	Finish loading and choose Start/Continue conversation. Click the Game view, check the connection and microphone, and release the VR confirmation button before pressing it for recording.
Wrong language / voice	Set 03 LANGUAGE and click the intended name in 04 VOICE. A play-icon click only previews. Speak in the selected language.
No sound / preview times out	Check output, volume, Game-view audio and the backend connection. Click play again. Late cancelled previews are ignored.
VR exits immediately	Stop Play, close Unity, enter Quest Link first, then reopen Unity. Check OpenXR and headset audio settings.
Part of an answer is missing	The scene has a 60-second queued-audio limit. Long replies can overflow it. Check Console errors and complete playback.

Setup is complete when

The GPU check passes, Unity starts without compiler errors, loading blocks input, the menu works, and both selected languages and voices play complete answers. Check text boxes, lip-sync, gaze and interruption with a real microphone. Repeat in the headset if needed. Test offline separately before relying on it.